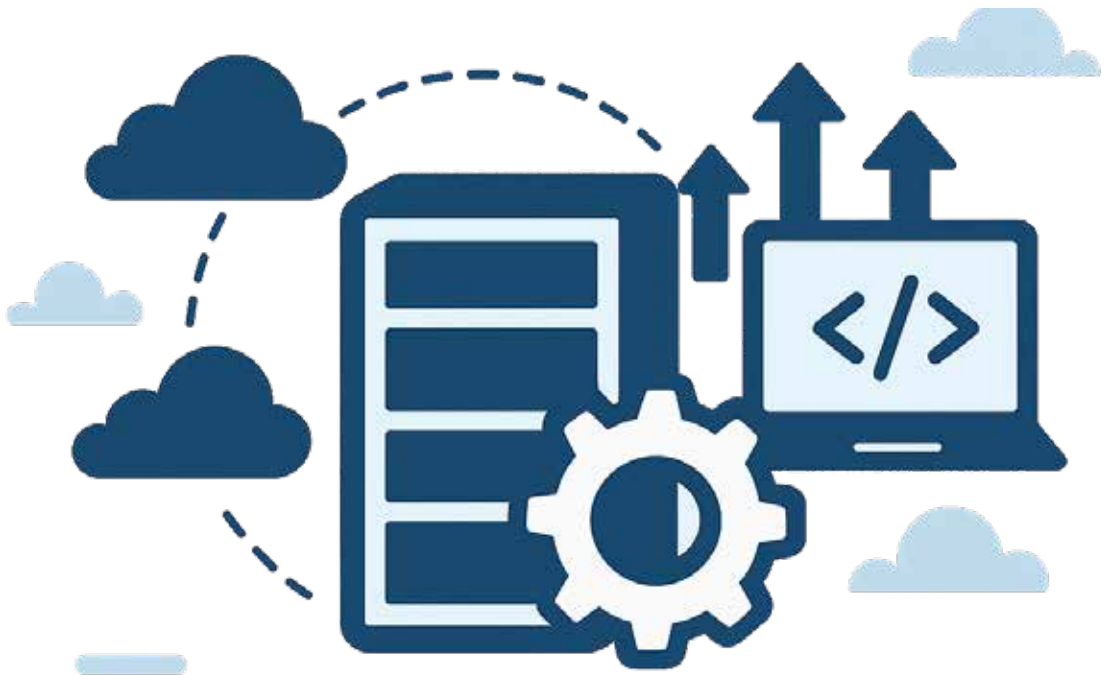
Serverless Containers: Exploring Emerging Trends

Over the years, virtual machines and traditional containers have made way for serverless computing and serverless containers. Here's a quick look at the major platforms offering serverless containers, the benefits and trade-offs of this technology, and where it's headed.



Ten years ago, if you were building a software application, you would likely run it on a virtual machine. You'd need to install the operating system, allocate resources and maintain the server environment. This worked fine, but it was cumbersome, resource intensive and costly to manage. Then came containers — smaller, faster and more streamlined.

Fast forward to today, and the buzzword is 'serverless containers'. It sounds like a contradiction at first. How can a container — a system meant to run inside a server — be serverless? However, serverless containers are a logical development in the cloud computing space. Because they don't have to worry about the underlying server infrastructure, serverless containers let developers package their apps just like they would in a traditional container. Cloud providers handle that for you. You simply focus on writing your application and uploading the container. The cloud runs it, scales it when needed, and stops it when it's idle. You pay only for the actual usage.

From containers to serverless

It's important to examine the evolution of software deployment in order to understand serverless containers. In the early 2000s, using virtual machines (VMs) to run applications was standard practice. A virtual machine (VM) allowed several 'virtual' computers to operate on a single physical server. This was a significant improvement over using a single app on each server. This transition was led by VMware. However, there was performance overhead associated with virtual machines. They required different operating systems for every virtual environment, took longer to start and consumed more memory.